Why Enveloping

Report on the Evaluation of Meta-transaction Proposals

by: Sergio Lerner
Date: 13-JULY-2020
Rev: 1

Introduction

Meta-Transactions

Artificial Sender Replacement (ASR)

Multi-Signed Transactions (MST)
- RSKIP138
- EIP 2711

Dual Smart-Wallets / Counterfactual Creation (CC)

Account + Contract unification (EOC)
- RSKIP167

A Meta-transactions System for RSK

Meta-Transaction Evaluation Criteria

Summary

Introduction

Most blockchains have a preferred cryptocurrency to pay for transaction fees. This simple design has many benefits. First, to bootstrap an economy, the native token model creates an initial demand for a new token. Second, it simplifies the interaction between users and miners because it forces them to use the same means of payment. Third, it reduces the

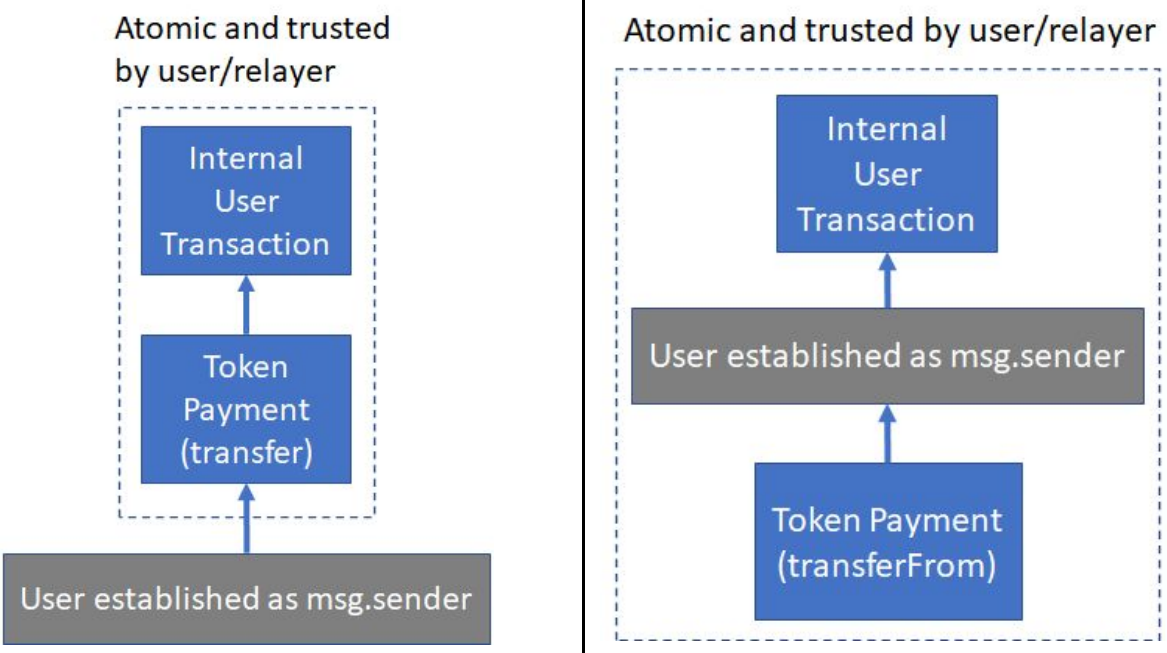
complexity of the consensus rules. Last, it provides DoS protection to the network as full nodes can foresee miner's wishes to include a received transaction. This way nodes can decide to propagate a transaction or not, preventing the free consumption of network bandwidth, and stop spam transactions. But with the advent of DeFi, several fiat-pegged tokens have become a preferred means of payment and savings for both users and miners. Therefore separate systems were created to facilitate this. Transactions that enable this functionality were named meta-transactions, because in some systems the user transaction is embedded in a higher-level (or meta) transaction created by a sponsor. A more clear term for these transactions are "envelopes" or, for the whole system, an Enveloping system. A meta-transaction/enveloping system can serve at least two different use cases: 1) pay the transaction fees with tokens, where one new party receives the tokens and pays the gas for the user, and 2) allow contract developers to subsidize the gas used to call their contracts. I wrote one proposal to provide enveloping functionality to RSK in 2017, but as many different solutions evolved, recently I began to explore the existing meta-transaction solutions to find the best one that could be deployed on RSK. At the same time, I wanted to find a solution that could be integrated fast into existing mobile wallets and one that could be compatible with pre-existent contracts that check msg.sender. Last but not least, the solution needed to be very cheap for the users in terms of the gas overhead.

In this article I will present a short update of the state of Meta-transactions and focus on the most promising solutions. I also introduce a criteria to evaluate and compare the different solutions objectively. But before, I'll first describe how meta-transactions work and what are the minimum technical requirements.

Meta-Transactions

The main use case for a meta transaction system is to let users pay transactions in tokens. This is done via intermediaries, often called **Relayers**, that are willing to receive the tokens and execute the transaction on behalf of the user. Some cases the meta-transaction solution also allows the user to negotiate a Quality-of-service (QoS) with the Relayer, for example, the ability to pay less for a longer transaction confirmation period.
For any meta-transaction system there is a point we call "sender establishment" where the transaction acquires the identity of the user (and not the one of the Relayer). This occurs either initially or along the execution path of the transaction. Both the user and the relayer want that a token payment to the Relayer and the execution of the user internal transaction be atomic, so the parties can be sure that either both operations happen, or none of them. In turn, the payment of tokens by the user to the Relayer can be performed from the user wallet (the ERC20 transfer() method), or can be delegated by the user to the Relayer, which will execute a transferFrom() method. Delegation requires a previous approval of unlimited token transfers (the approve() method of the ERC20 interface) to the Relayer. If the token payment is performed from the user account (transfer()), then the system must allow atomicity of two operation after the sender establishment (transfer() and then execute user code). If the token payment is delegated, then the system should at least allow atomicity of two operations prior

to a sender establishment (transferFrom() and then execute user code with new sender). In the last case, the code that is executed before sender establishment must be trusted by both parties. This implies that either the user must establish a trust relationship with the Relayer before using the system or the user must establish a trust relationship with a contract that all Relayers trust, and this last option is the simplest. The pre-approval method works with ERC20 tokens, and also for EIP721 NFT's, but not all transferable assets may support delegation. In fact many people think that delegation should not have been part of the token standard. For assets that do not support transfer delegation, the solution would be to force the assets used for payment to be stored in a trusted intermediate custodian contract. Here we show the two options regarding atomicity and sender establishment schematically:



(a) Atomicity post msg.sender establishment

(b) Atomicity pre msg.sender establishment

Arrows indicate only execution order (origin executes first). Dashed rectangle indicates code that needs to execute atomically and must be trusted by all parties.

Now we're ready to discuss the different existing solutions. All existing solution fall into one of the following distinct classes:

- **Artificial Sender Replacement (ASR)**: requires sender establishment in some point the execution path.
- **Multi-signed transactions (MST)**: initial sender establishment
- **Dual Smart-Wallets and Counterfactual Creation (CC)**: standard sender establishment with internal transaction (CALL opcode)
- **Account + Contract unification by Externally Owned Contracts (EOC)**: standard sender establishment with external transaction

Artificial Sender Replacement (ASR)

Artificial sender-replacement solutions provide an opcode or precompile contract that performs a call on behalf of another user, embedded in normal bytecode execution flow. You can think about them as using a Unix “sudo” call in a batch file or as a “RunAs” in Windows, but instead of providing a password, a relayer provides a signed message by a grantor (which is the end-user) to authorize the use of the grantor’s credentials. However, in the context of the EVM, the grantor’s credential involves only the change in the msg.sender field for a certain grantor’s code payload (generally a CALL or code snippet). The executed routine doesn’t have access to the grantor’s account balance or storage, it only acquires the grantor’s msg.sender. The grantor’s message must also be protected from replay attacks. An example of a solution that performs artificial sender replacement is **CallWithSigner**. Sender-replacement proposals have to deal with the problem of replay protection. The easiest way to deal with it is by creating a new nonce queue for each user that is incremented in the embedded “sudo” calls. Sharing the internal “sudo” nonce queue with the standard nonce queue is considered a bad practice because it may create nonce conflicts with external transactions waiting in the mempool, and this can generally be abused by an attacker to flood the network with long transactions without paying the bandwidth costs. Artificial signer-replacements solutions also have to deal with the payload format that must be signed, and are inflexible to any change in such format. It is expected for a well-designed proposal to define the signed message conformant to EIP712 to prevent type-confusion attacks to smart wallets, but currently CallWithSigner does not comply with this EIP. Another example of artificial sender replacement is EIP1035. RSKIP166 is another variation of this concept, but shares a single nonce queue for external and internal replay protection, which complicates mempool handling, and uses a precompile contract instead of an opcode, which removes the need to modify the Solidity compiler.

Multi-Signed Transactions (MST)

Multi-signed transactions define a new transaction format that involves signatures from multiple parties, where the transaction data needs to be agreed by all of them. One of the parties, usually called the sponsor or relayer, is optionally given the role of paying the full transaction fee. Bitcoin is a classical example of multi-signed transactions. In case of colored-coins, the user can provide colored-inputs and a sponsor can provide the bitcoins to pay transaction fees, and all participants sign the full transaction (SIGHASH flags can improve the solution by reducing the number of interactions required to sign). Neither RSK nor Ethereum allow atomic transaction batches, so such a system must be introduced as a consensus change. The nice property of Multi-signed transactions is that the sender establishment is performed initially (there is no switch), and therefore there are no problems that can affect mempool safety, although multi-signed transaction require more intelligent

algorithms to decide when to invalidate mempool transactions, and when fee-bumping is allowed.

RSKIP138

RSKIP138, is a consensus change to allow multi-signed transactions for RSK, and therefore one that can also be applied to Ethereum. RSKIP138 adds to the consensus a new type of transaction that supports multiple signers, each providing his own nonce and signature. The proposal optionally let the last signer pay for all the transaction gas. It's easy to see how multi-signed transactions enable meta-transactions: let's suppose that the sender wants to call a contract, and the sponsor pays the transaction fees. Both would sign an agreed bytecode payload that includes a token payment from the user to the sponsor and later the user-chosen call. The RSKIP138 has some additional benefits, such as simplifying payment channels (and other multi-party protocols) where many parties agree on a state change. Even if RSKIP138 defines the transaction format to be highly compatible with existing wallets, and even with some hardware-wallets, it's not a simple change. Also, to serve for meta-transactions, RSKIP138 must be combined with a variant of the rich-transactions proposal to achieve atomicity, this is because multi-signed transactions perform the sender establishment initially, so that atomicity must come later. In RSKIP138, the bytecode in the "data" field of the transaction is atomically executed in the context of the user EOC.
RSKIP138 also adds some other features, such as native multi-party accounts, that can help in the creation of cheaper and safer multi-party protocols, such as payment channels.

EIP2711

<content temporarily removed from here. It will be re-included after some corrections >

Dual Smart-Wallets / Counterfactual Creation (CC)

The dream of secure, generic and easy to use smart-wallets has been around since the creation of Ethereum. Smart-wallets are savings accounts that let the owner restrict the allowed operations by means of contract code logic, in order to protect the user from mistakes, coercion, hacks, lost private keys, and other operational risks. This is indeed the direction that the cryptocurrency ecosystem is moving. With the advent of the CREATE2 opcode, it's possible to use counterfactual contracts to receive tokens and spend them, without having to deal with native currency. Some mobile wallets like Argent and Gnosis Safe already use smart-wallets underneath. Under this model, the user wallet is split in two: one Externally Owned Account (EOA) to store native currency (RBTC for RSK and Ether for Ethereum) and a wallet contract that stores tokens (eventually also native currency) and embeds additional rules to enhance the security of the wallet. The idea to achieve meta-transactions under this model is that the wallet contract is counterfactually created. It can receive tokens and only when there is a need to spend them the client decides either to

issue a transaction from the EOA having native currency to pay for gas or, if it doesn't have it, to use the meta-transaction system. To enable meta-transactions, the wallet contract is created supporting a method execute() that is capable of receiving a transactional payload signed by the user, which gets executed in the context of the wallet contract after both signature and replay protection mechanisms are checked. Later we'll see that this functionality is essentially what is provided by the Gas Station Network's Forwarder contract. This approach has the huge benefit that it doesn't require any consensus change. The main problems of this approach are:

- The dual model and counterfactual creation of wallet contracts complicate the life of wallet developers.
- In the simplest implementation, users must manage two different addresses, one for storing native currency and another for storing tokens.
- There is a moderate initial cost to deploy the contract counterfactually (about 70K gas with highly optimized code).

Account + Contract unification (EOC)

Smart-wallets is something that has been advertised since 2016 to be "around the corner" by the Ethereum community. Therefore we might analyze why the revolution of smart-wallets was delayed for 4 years. It seems that the inexistence of counterfactual contract creation prior February 2019 (the Constantinople hard-fork) may have been one reason. But still another reason might be that the dual model (an EOA to hold some amount of RBTC/Ether to pay fees and a wallet contract to hold the rest), is inherently difficult for wallets to implement safely and for users to understand. So maybe a simplified native smart-wallets model is the catalyzer the ecosystem needs for the paradigm shift.
The simplest possible implementation for smart-wallets is using Externally Owned Contracts (EOCs). EOCs combine contract code with Externally Owned Accounts (EOAs). EOCs have the ability to execute commands sent to them, and also perform calls from transactions that originate from the EOC address using a standard ECDSA private key and signature. Most contracts in RSK/Ethereum are EOCs, but controlled by an unknown private key, but because it is infeasible to find the matching private key, for all practical purposes externally owned accounts (EOAs) and contracts work as separate entities over the same address space. EOCs are related to the concept of Account Abstraction (or AA), but these two concepts are different. EOCs do not abstract the signature verification logic for transactions originating from the EOC address, nor it allows this logic to be extended or removed. AA goes one step further by removing the concept of signed transactions from the consensus layer. Both may enable meta-transactions, but AA also has to deal directly with a new logic to govern the mempool, preventing DoS attacks, and to provide replay protection for transactions. One of the first goals of Vitalik was to come up with an AA proposal that would be compatible with Ethereum 1.0, but the set of changes required seems to be too large, and the risk too high, to attempt a migration. Ethereum 2.0 will try to revert this flaw, and the official Ethereum 2.0 documentation states:

> “Ethereum account abstraction has the goal of reducing from two account types down to one, a contract account. The single account type will have the functionality to transact both coin and contract. Developer and user will no longer need to make a distinction between account type since transacting will be moved fully into the EVM and off of the blockchain protocol."

There is a project that has implemented AA on Ethereum 1.0 here.
A good EOCs proposal should not need to deal with mempool safety, and therefore would be much more palatable for inclusion in RSK or Ethereum than AA.

RSKIP167

RSKIP167 is a minimalistic attempt to unify accounts and contracts into EOCs. It introduces a new precompiled contract that allows a user (or a sponsor) to deploy code into an existing or new EOA. To deploy code, the signature of the owner of the EOA is verified, over a message that is formatted according to EIP191 (or it could also be EIP712) to avoid type-confusion attacks between normal RSK/Ethereum transactions, EIP712 transactions, and contract deployments. RSKIP167 was designed to be implemented with a short and concise code (it required less than 200 lines in our reference implementation). One key decision made was to avoid initialization code execution. The code provided is immediately installed in the target account and the code must use other initialization methods (if required) to be fully configured. The reason is while loading code into another account is somehow an anomaly of code flow, performing an initialization step just worsen the anomaly, as no Ethereum precompile currently calls another contract. It’s also important to keep the code installation cost as low as possible, as this will be a one-time cost that must be paid by any user willing to use the meta-transaction system. For the user to begin using the system, a transaction is created that combines atomically instructions to install the code and to call the code so that the call pays back tokens to the sponsor.
All meta-transaction solutions based on EOCs would require this initial step, and this step can be problematic if the sponsor lets the user install arbitrary code. For example, the user could provide a code to install that does not enable paying tokens to the sponsor. By offering only a limited gas for code installation, the sponsor could accept that risk to serve malicious users, and ban the malicious user account for further service requests. Depending on the consensus change we apply, code installation can be more or less expensive.

The order of these two actions for code installation is the reverse as all remaining meta-transactions. In normal meta-transactions, first the user pays the tokens to the sponsor and then the user transaction is executed. This ensures the sponsor always gets the tokens. If the order were reversed, the user transaction may double-spend the tokens before the sponsor is able to get them. In some systems a third step is executed where the sponsor reimburses the user with tokens equivalent to unused gas.
In case the sponsor cannot bear the cost of malicious code installation, how does the sponsor ensure that the user will pay for the installation when installing arbitrary code?
The sponsor could check off-chain that the code provided by the user conforms to a standard template which contains an unavoidable code path to pay the tokens.

One of the main objections to RSKIP167 is that it applies retroactively to any EOC. If an owner of the EOC has already signed a message that complies to the EIP712 specified, then the account could be rendered insecure. This is however highly improvable. Another objection is that the attacker manages to convince the user to sign a single specially-crafted EIP712 message, then the attacker can install a “backdoor” code on the wallet that can be used long time in the future to steal user tokens. However, such a backdoor cannot be hidden, and the wallet can detect and alert immediately of any modification of its own code. This “backdoor” is no different from the existent capabilities of contract-based wallets to install custom verification modules (present both in Gnosis Safe and Argent wallets). In fact, Gnosis Safe Modules have shown to be able to work as such. Therefore, if we penalize the existence of this flexibility, we must apply the penalization to other wallets as well. If we’re really paranoid of this risk, then it is also possible to modify the InstallCode precompiled contract to restrict the codes that can be deployed and enforce a certain preamble in the code. This preamble would force an initial call to a predefined Solidity contract (SigVerify) that performs signature verification and provides replay protection. Therefore, we remove the risk that the user, by mistake or by a phishing attack, deploys a malicious code into his account. The preamble would still force signature verification for all external commands. In the future, when wallets have been thoroughly tested and users are more used to interact securely with EIP712-based signed messages, this code preamble restriction could be lifted.

A Meta-transactions System for RSK

Achieving a fast time-to-market usually means reusing existing components, but it also generally implies inheriting the main properties of the reused system (both advantages and disadvantages). Currently the only standardized and deployed meta-transaction system is the Gas Station Network. So I decided to find the minimum set of changes to the GSN that would reduce GSN overhead and provide compatibility with old contracts. If you look at how GSN works, there is a single critical interface called IForwarder that practically defines how the whole solution should work. A Forwarder is a contract that publishes a method to execute transactions in its own context, where the transaction payload is signed by a grantor. The Forwarder verifies the signature and prevents replay attacks, and then continues with the execution of the payload. In GSN, contracts must trust a Forwarder and query the Forwarder for the right msg.sender. The Forwarder returns the signer of the payload as the sender, instead of the Forwarded contract itself, which is what msg.sender returns. To reuse the GSN, one only has to rethink which contract will support the IForwarder interface.
Let’s recall, even if we reused GSN components, we still need to be compatible with older contracts that do not support querying a Forwarder for the correct msg.sender. Depending on the type of native meta-transaction consensus change, there is a corresponding change to be applied to GSN Forwarder:

- For Artificial Sender Replacement, a new version of the GSN Forwarder performs the sender replacement internally using the grantor's signature, and finally calls the destination contract with a replaced msg.sender. The Forwarder is a singleton contract trusted by all parties involved.
- For a Dual Smart-Wallet, the wallet contract is enhanced to implement the IForwarder interface. The wallet contract would receive commands from its owner wrapped in third-party transactions coming from Relayers. Each user wallet must implement the IForwarder interface, but all of them can rely on a single library contract to share the Forwarder functionality.
- For Account / Contract Unification, the same happens as for the Dual Smart-Wallet system, but now it's the EOC the contract that implements the IForwarder interface.

Meta-Transaction Evaluation Criteria

To come up with the best meta-transaction solution, I needed to research into the benefits and downsides of MST vs CC vs EOCs vs ASR. To do that, I created both viable EOC and MST proposals, and I estimated gas costs for a CC solution. For ASR, I use CallWithSigner proposal. Now I needed a criteria to compare them, and therefore the meta-transaction evaluation criteria was born.

The criteria attempts to capture the fitness of a meta-transaction solution for a particular use case I'm interested in: financial inclusion. This use case is characterized by millions of users having simple wallets that mainly hold one or two tokens, generally stable coins, and performing either gas-less payments to other users or interacting with 2nd layer payment solutions such as zkSync, at a lowest possible cost.

The criteria is split into requirements, but not all requirements are equally important. We present one possible prioritization of the requirements using scores (shown in brackets). A meta-transaction solution may not satisfy all the requirements, so we search for solutions that can reach a high score.

1. **Main use case**: Allows the owner of an account that does not have native currency (RBTC/Ether) or does not want to spend their native currency, to perform a owner-approved transaction. [infinite]
2. **Security**: The solution should not allow any party to steal user or Relayer funds. Neither the solution should allow low-cost DoS attacks to users or Relayers. Also the solution should not open new vectors of attack to the platform [50]
3. **Compatibility with pre-deployed contracts**: the solutions should be compatible with all non-malicious smart-contracts previously deployed. It may be incompatible with a contract specifically designed to be incompatible with the solution, which we'll assume malicious. To be compatible, the sender of the transaction returned by msg.sender must match the user account itself [35]

4. **Gas Efficiency**: In its more efficient form can perform the main use case consuming less than 40K gas on average. We allow this value to take into account amortization over 10 transactions with a form of batching, if possible. Additional costs of optional QoS arbitration are not accounted for. [30]
5. **Reduced time-to-market for client-server code**: It should be possible to borrow a client-server interaction code from another Meta-transaction solution to reduce time-to-market, or the interaction must be so simple that it can be developed in less than 2 months. [30]
6. **User protection using Relay Penalizer**: Allows the use of a Penalizer system to prevent double-spends by Relayers [25]
7. **Cheap enough for working MVP**: We request that the average gas consumed in the first release of the solution is lower than 100k (and that later it can be improved to reach the first target) [20]
8. **Transaction Execution Simplicity**: An initial working system that can be implemented based on the proposed solution should provide a simple flow during meta-transaction execution: (1) no more than 4 inter-contract calls, and (2) no more than 3 storage cells modified. We also request that consensus changes should not take more than 300 lines of code, and Solidity code involved not more than 300 lines. This does not preclude that future expansions of the system transform it into a more complex system to serve other use cases. [20]
9. **Gas Fairness**: It should allow for the reimbursement of unused gas. [20]
10. **Signature Segregation**: It should be possible to expand the solution to support segregation of witness data. In other words, the signature must not be stored in contract storage. [15]
11. **Compatibility with Native Wallets in Transaction Format**: The solution can be made compatible with existing wallets, including hardware wallets, for signing the meta-transaction payload. [15]
12. **Compatibility with Meta-Transaction Wallets**: The solution can be made compatible with wallets that use another Meta-transaction solution to facilitate port to the new system. [15]
13. **Additional Core Features:** The consensus changes may provide additional features that are interesting to fulfil completely different use cases. [15]
14. **Compatibility with Future Ethereum Upgrades**: high probability to be compatible with future Ethereum upgrades for native meta-transactions or account abstraction. It's important that the solution does not break code flow analysis tools, EVM debuggers, Remix online IDE, and the truffle tool. [15]
15. **Allows a QoS Arbiter**: Allows to trade price for longer waits using a QoS Arbiter. [15]
16. **Sponsor Contracts**: Enable other secondary use cases, such as that a destination contracts pays the gas for the user. The maximum score is given if the use case is already supported by reusing code. [10]
17. **Fairness 2**: It should allow for transferring to the user the EVM gas reimbursements due to cleared storage cells and contracts self-destroy. [10]
18. **Allows Relay-side batching**: Relayers may want to accumulate several transaction payloads from different users into a single external transaction to amortize the base cost of transaction inclusion (21K). : [10]

19. **Allows user-side batching:** Allows users to include several different contract calls into a single transactions to reduce costs [10]
20. **Onchain exchange**: Enable Relayers to accept tokens but exchange them for RBTC immediately and atomically in the same transaction. The maximum score is given if the use case is already supported by reusing code [10]
21. **Implementation of consensus changes**: The consensus changes have already been implemented and tested. [10]
22. **Prevent Relayer transaction double-spend**: If the solution blocks the Relayer from making more transactions (the relayer cannot continue serving other users without forcing a transaction to be included in a block) then the solution should allow unilateral fee-bumping by the Relayer [20]
23. **Existing the solution**: It could be beneficial to be able to easily terminate the system if another superior meta-transaction standard arises and if the consensus change does not provide any additional positive functionality. [10]
24. **Quality of Specification**: under-specified proposals have higher risk of hiding problems. Low and high level specifications allow different persons to evaluate different parts of the proposals. [10]
25. **Transaction Verification Transparency**: Making users sign opaque transaction data to interact with an app is unfortunately a common problem. Transactions sending native coins and some ERC20 specific tokens are displayed correctly by some resource-limited hardware wallets, allowing users to verify what they are signing in common use cases. A meta-transaction solution that does not break currently working transaction verification systems (i.e. does not require changes in hardware wallet firmwares) would be more secure for end users. However since the main use case is tailored for users using mobile wallets, hardware-wallet is not our main concern. Our use case targets users that perform low/mid value transactions. Users who perform high-value transactions won't probably need meta-transactions at all. Still, a meta-transaction system that conforms with EIP712 allows transaction verification with higher security, and therefore are considered to verify this criteria item [10]
26. **Mempool safety**: The solution does not break mempool invariants regarding nonces. If the solution requires to update the mempool on events occurring while executing a block (apart from the inclusion of transactions per-se), then this can bring new efficiency bottlenecks and unknown security risks. [15]
27. **Does not require consensus change:** The solution achieves the main goals without requiring a consensus change.[30]
28. **Compatibility with Existing Wallets in Account Derivation format:** some solutions require the modification of wallet clients and servers to adapt to new ways to derive account addresses. Solutions that can re-use the current address derivation system are preferable. [15]
29. **Initial cost**: per-wallet deployment cost should be lower than 100K gas. To compute this cost, base transaction costs (21K) can be amortized in 10 batched deployments, if batching is allowed. [10]
30. **Single Account**: Dual-wallet solutions usually require that the user is aware that they own two different addresses, one for storing native currency and another for storing tokens. This can lead to human error, where the user or a payer sends tokens to the

EOA instead of the contract, and the tokens become either locked until the user refills the EOA with native currency or they are temporarily stored insecurely because they are out of reach of the wallet contract protection logic (i.e. whitelists, rate limiter, etc.) [10]

31. **No change to the Solidity compiler**: Changing the Solidity compiler is problematic since many blockchains, including RSK, Ethereum and ETC, use the same compiler. It also forces contracts to upgrade. [10]

The following table shows the results obtained according to our evaluation criteria:

	CallWithSigner	RSKIP138	RSKIP166	RSKIP167	Dual-wallet
Technique	ASR	MST	ASR	EOC	CC
Score	390	405	325	447	448

Summary

We presented the different existing approaches to enable a meta-transaction system based on Relayers that are backward compatible with contracts already deployed in the RSK and Ethereum blockchains. We created a list of desired properties of a satisfactory meta-transaction solution and assigned each of them a score that approximately represents the value each property brings to the platform. Finally we ran our scoring criteria against some selected candidates, covering the different approaches to meta-transactions. The result shows that both the native approach based on dual-wallets (counterfactual wallet contract creation) and the approach based on external owned contracts can be the basis of a meta-transaction solution that can be designed, developed, and deployed in a reduced time over RSK. Because of the close score ties between the two selected proposals, and in order to make a final decision, either the criteria scores need to be reevaluated or other properties not currently considered in the criteria must be added in order to differentiate them. If no more time would be used in the system design phase, then Dual Wallets should be chosen as it reduces unforeseen risks.